SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 13.2

Program	**: B. Tech (CSE)**
Specialization	**: --**
Course Title	**: AI Assisted coding**
Course Code	**:**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: Akash Reddy**
Enrollment No.	**: 2403A51L30**
Batch No.	**: 51**
Date	**:10-03-2026**

Task Description -1 (Refactoring – Eliminating Repetitive Logic):

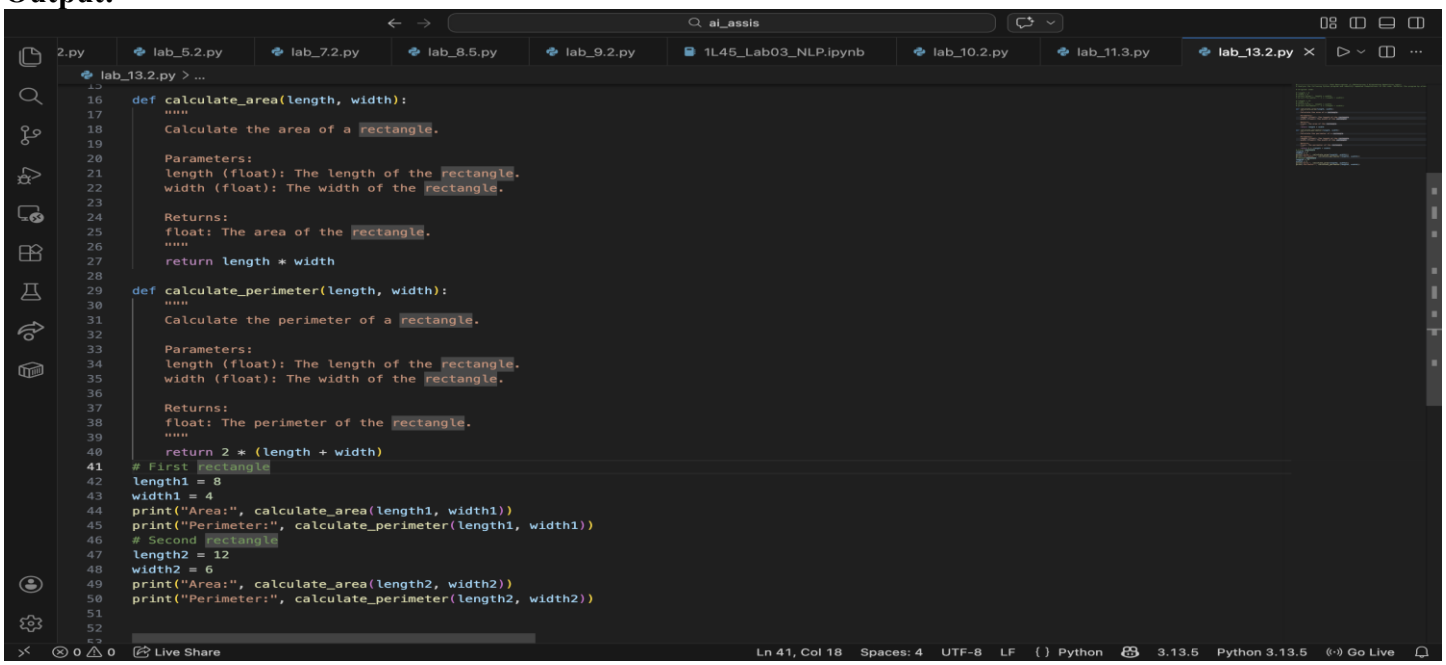
Prompt: Analyze the following Python program and identify repeated computations in the code. Refactor the program by eliminating duplicated logic and encapsulating the repeated calculations into a reusable function. Ensure that the output of the program remains exactly the same after refactoring. Also add clear and proper docstrings to all defined functions explaining their purpose, parameters, and return values.

Original Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Output:



```
/usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
akash@AKASHs-MacBook-Air ai_assis % /usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
akash@AKASHs-MacBook-Air ai_assis %
```

Task Description -2 (Refactoring – Modularizing Inline Calculations):

Prompt: Analyze the following Python script and identify repeated inline calculations related to discount and final amount. Refactor the program by extracting these repeated calculations into a reusable function. Ensure the program becomes more modular, readable, and maintainable. The refactored function should include proper docstrings explaining its purpose, parameters, and return values. Also verify that the output of the program remains the same after refactoring.

Original Code:

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Output:

```
def calculate_final_amount(amount):
    Calculate the final amount after applying a 10% discount.

    Parameters:
    amount (float): The original amount before discount.

    Returns:
    float: The final amount after applying the discount.
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount
# First amount
amount1 = 1000
print("Final Amount:", calculate_final_amount(amount1))
# Second amount
amount2 = 2000
```

```
/usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
akash@AKASHs-MacBook-Air ai_assis % /usr/local/bin/python3 /Users/akash/Desktop/ai_assis/lab_13.2.py
Final Amount: 900.0
Final Amount: 1800.0
akash@AKASHs-MacBook-Air ai_assis %
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance):

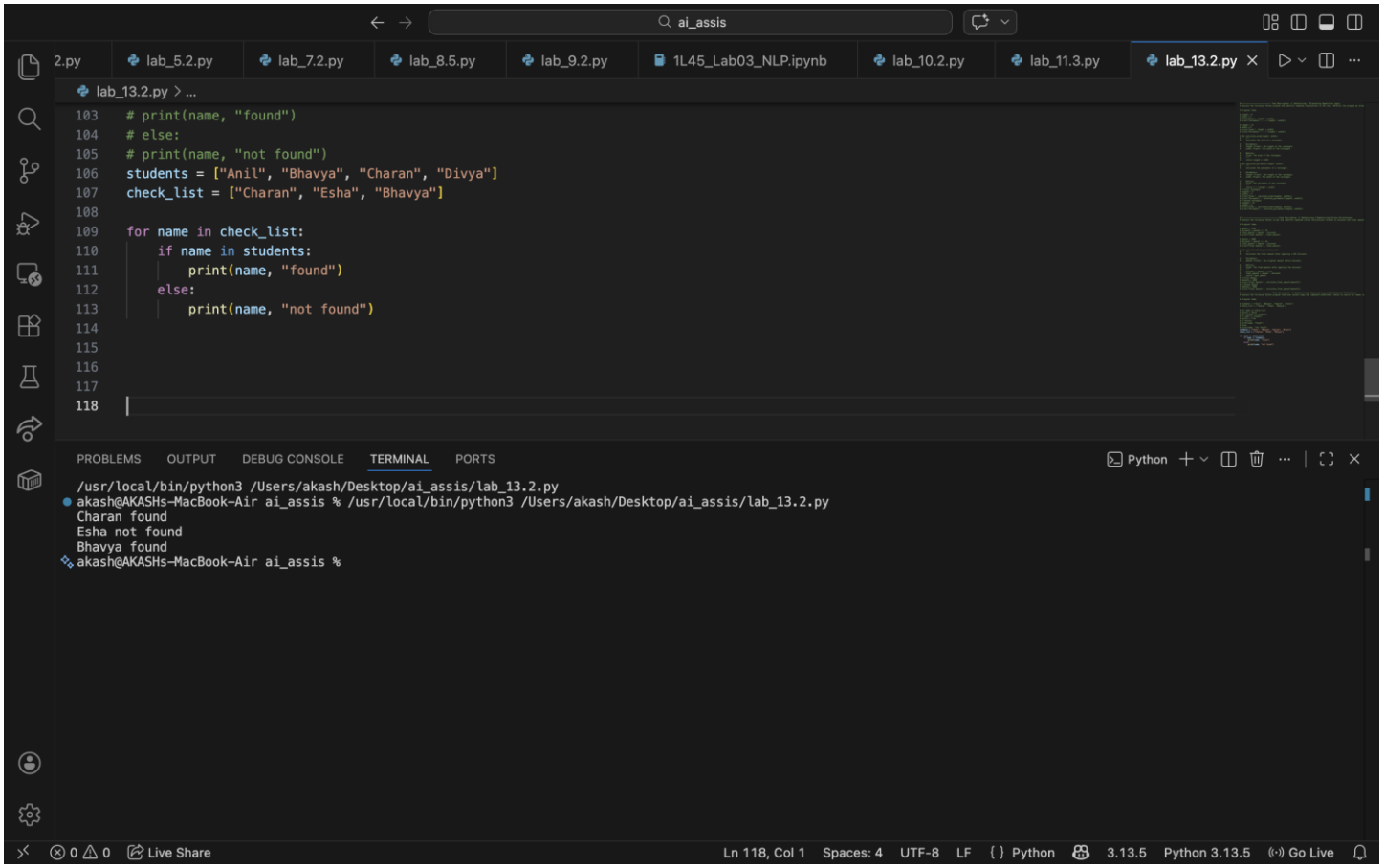
Prompt: Analyze the following Python program that uses nested loops and repeated conditional checks to search for names in a list. Identify performance issues in the code and suggest improvements using more efficient data structures or simplified logic. Refactor the program to replace inefficient constructs while preserving the original behavior and output. Additionally, compare the execution efficiency of the original and optimized versions of the code and explain the performance improvement. Ensure the refactored code remains readable and well-structured.

Original Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
else:
print(name, "not found")
```

Output:



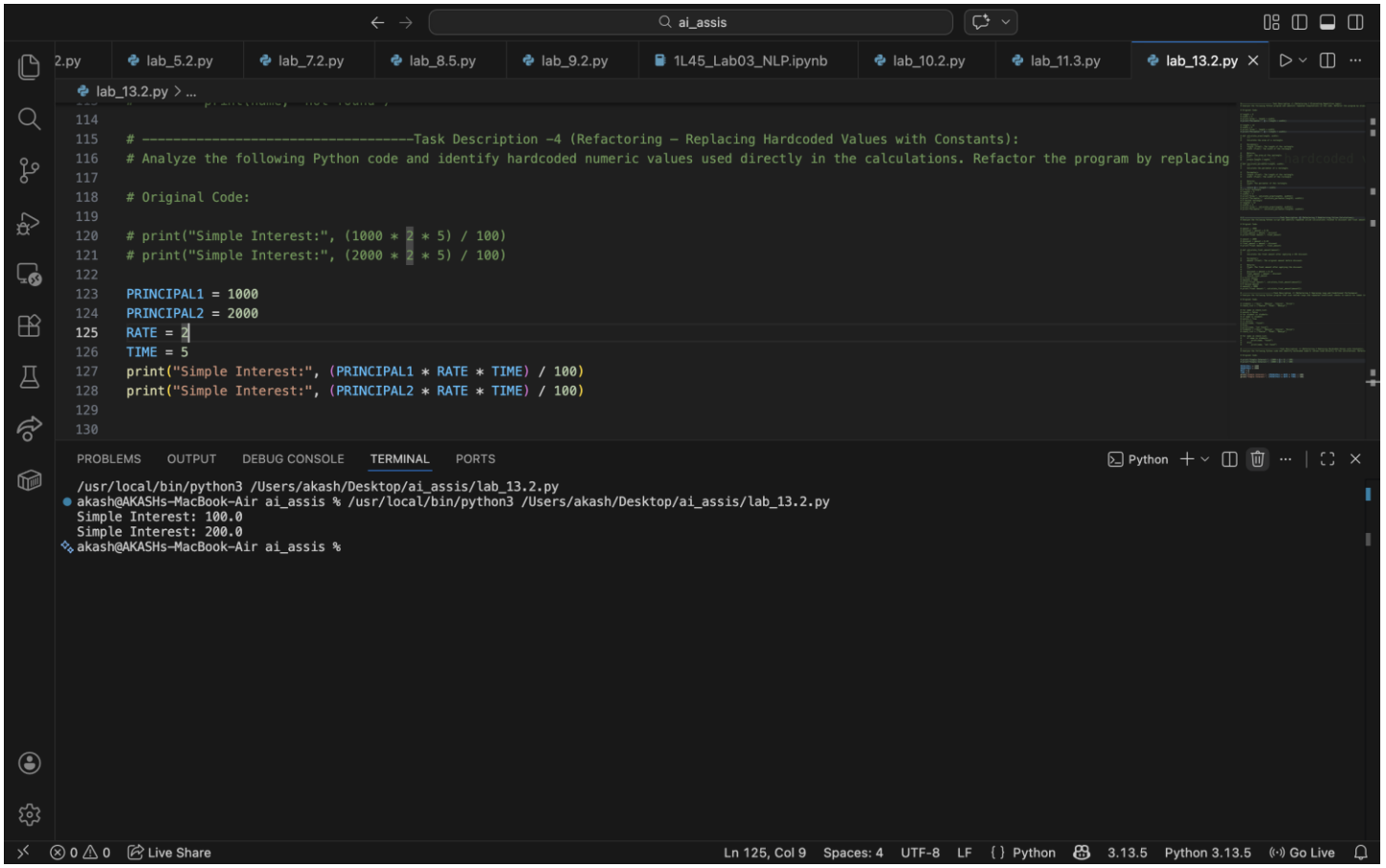
Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants):

Prompt: Analyze the following Python code and identify hardcoded numeric values used directly in the calculations. Refactor the program by replacing these hardcoded values with descriptive named constants defined at the beginning of the program. Ensure the constants clearly represent their meaning (for example, RATE and TIME). Update the calculations to use these constants while keeping the program output unchanged. Also ensure the refactored code is readable and maintainable.

Original Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

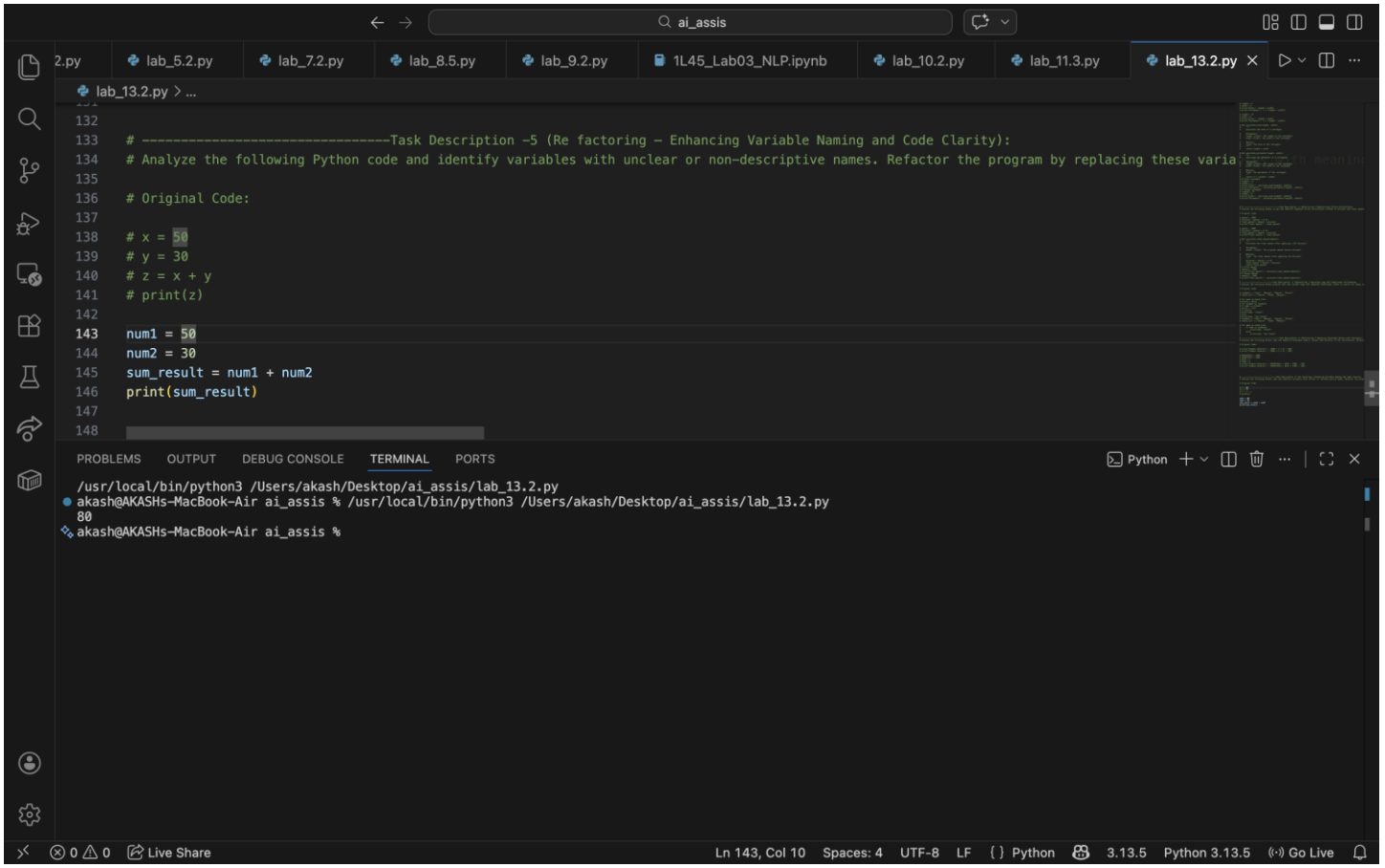
Output:



Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity):

Prompt: Analyze the following Python code and identify variables with unclear or non-descriptive names. Refactor the program by replacing these variables with meaningful and descriptive identifiers that clearly represent their purpose. Add inline comments where necessary to explain the logic of the code and improve readability. Ensure that the refactored program maintains the same functionality and produces the same output as the original code.

Original Code:

```
x = 50
y = 30
z = x + y
print(z)
```

Output: